

Healthcare Agent Platform

LangGraph Orchestration + MCP Tools

Architecture specification for training staff on protocols, CEO-approved policy publishing, draft prescription support, and refund decisions with mandatory human-in-the-loop.

Division of labour	LangGraph = how work flows · MCP = what the agent can do
HITL mechanism	LangGraph interrupt() + Postgres checkpointer + CEO / doctor resume
Quality plane	Observability + guardrails + evals + golden dataset + LangSmith
Tool access	Healthcare MCP server over Streamable HTTP via MCPAdapter
Clinical rule	Prescriptions are drafts until a doctor signs
Financial rule	Refunds execute only after CEO resume

Requirements mapped

- 1. Train doctors and clinical staff on treatment protocols.
- 2. PDF policy / training corpus (diabetes protocol, refund policy, and similar).
- 3. Anyone may propose a document update; CEO must approve before publish.
- 4. First prescription: medical reports + ~300 answered questions + doctor conversation. Later prescriptions: reports + a new conversation.
- 5. Refunds require HITL with the CEO.

1. Why LangGraph and MCP together

MCP and LangGraph solve different problems. MCP is the contract for tools, resources, and prompts — a USB-C port for capabilities. LangGraph is the durable flowchart: typed state, conditional routing, retries, subgraphs, and interrupts that can wait hours for a CEO or a physician. Combining them keeps the tool surface small and reusable while putting governance in the graph, where it can be checkpointed.

Concern	Owner	Why
Search protocols, read PDFs, draft Rx, file a refund	Healthcare MCP server	Stable schemas; reusable from any MCP client
Which specialist runs, what is remembered	LangGraph supervisor + subgraphs	Explicit control flow beats a single ReAct loop
CEO publish / CEO refund / doctor sign	LangGraph interrupt() + checkpointer	Park the run without holding an MCP socket open
Role injection, deny CEO tools to non-CEO	MCP adapter gateway	Defense in depth even if a node is mis-wired
PHI stores, versioned PDFs, notifications	Data plane behind MCP	Agents never talk to databases directly

Do **not** expose HITL graphs as LangGraph Platform MCP tools. Those invocations currently run as temporary runs without a durable checkpointer, so interrupt() cannot park and resume. HITL lives on the application API with durable threads. MCP is the tool plane the graph calls — not the place the CEO approves work.

2. Layered architecture

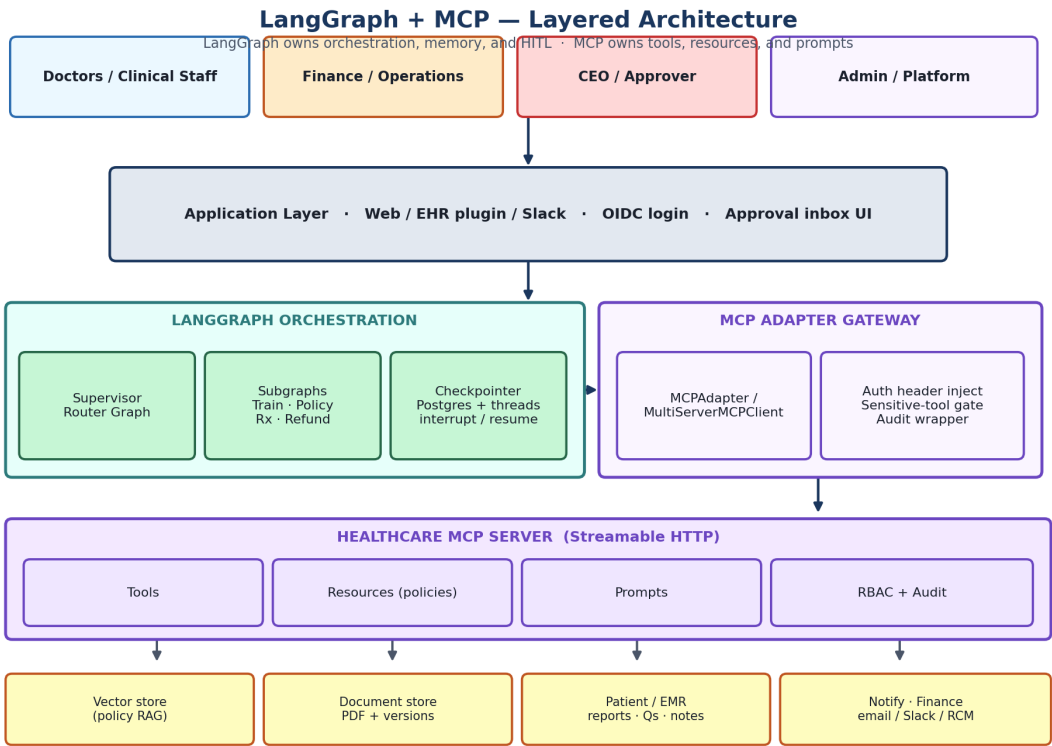


Figure 1 — Users and UI sit above LangGraph. LangGraph calls MCP through a gated adapter. The MCP server is the only process that touches stores.

Recommended transport for the Healthcare MCP server is Streamable HTTP on a private network. Stdio is fine for local development. The adapter injects the caller JWT so the MCP server can enforce RBAC per tool. Sensitive tools (review_policy_request, ceo_decide_refund, generate_prescription) are listed in a deny-unless-role map in the adapter as well as inside the server.

3. Supervisor graph and specialist subgraphs

A thin supervisor classifies intent with a constrained schema, then hands the same HealthcareState to one subgraph. Each subgraph is compiled separately so it can be tested and versioned on its own. The supervisor does not call MCP tools itself except a cheap FAQ search when intent is ambiguous.

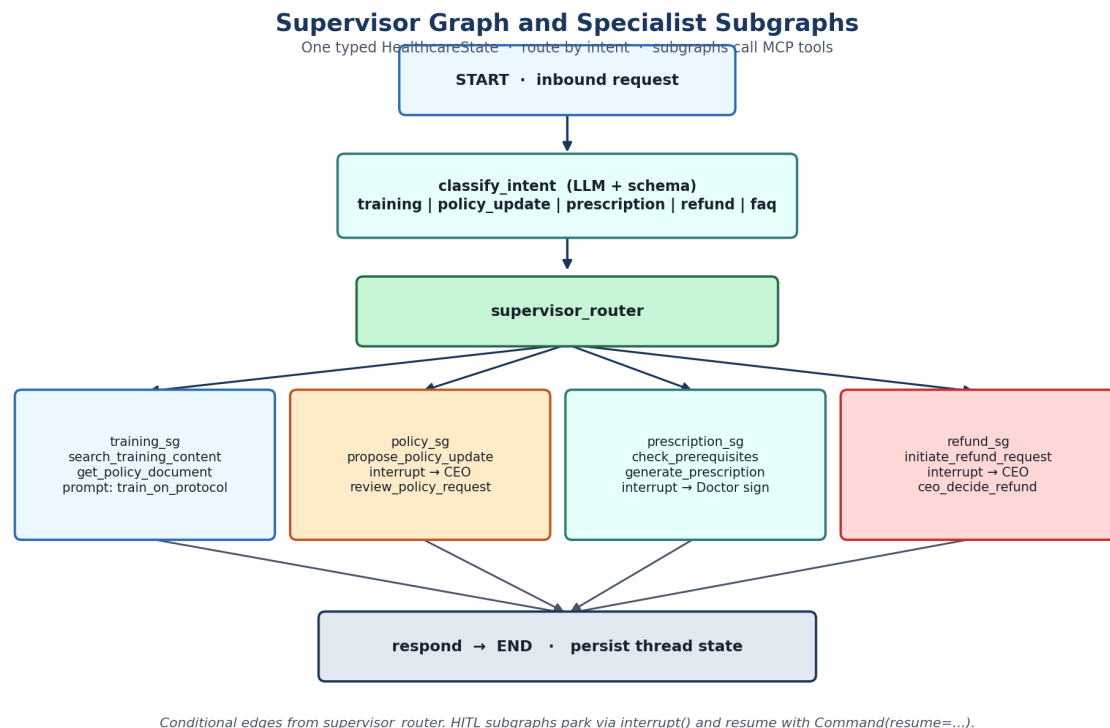
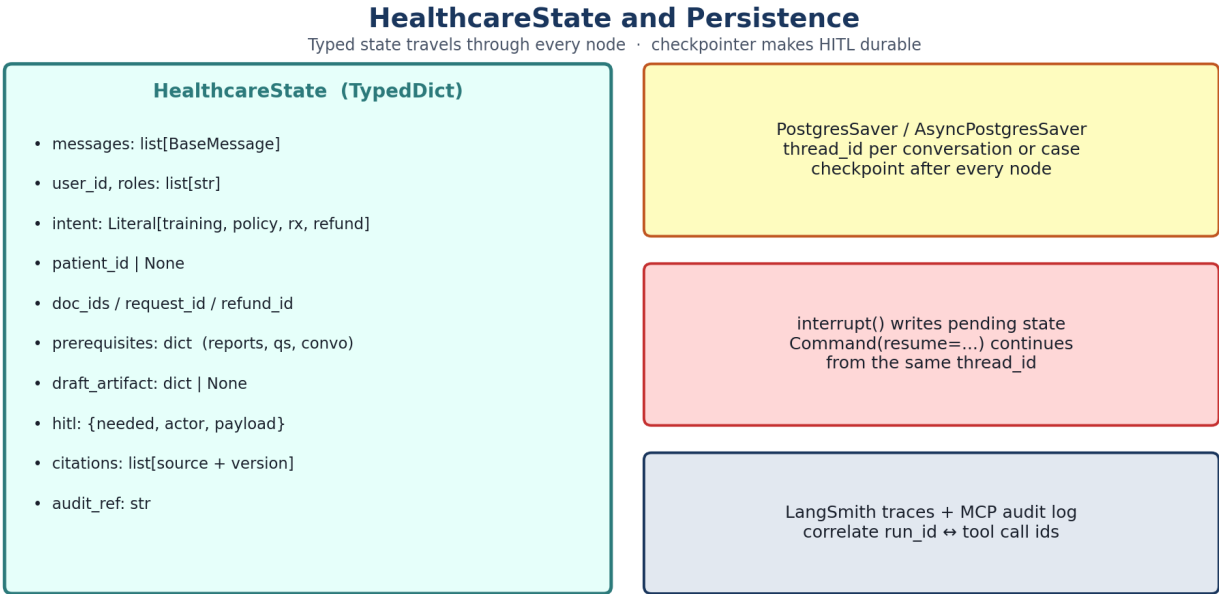


Figure 2 — Supervisor routes to training, policy, prescription, or refund subgraphs.

3.1 Intent routing

classify_intent returns one of: training, policy_update, prescription, refund, faq. Ambiguous utterances go to faq, which uses search_training_content and then may re-route. Do not let the supervisor become a second agent that re-plans every step — that duplicates the subgraphs.

4. HealthcareState and persistence



5. Training subgraph

Goal: walk a new doctor or clinician through the current published protocol (for example diabetes treatment) with citations that include document version. This subgraph is read-only against MCP and does not need an interrupt unless you add a competency quiz that a supervisor must sign off.

Nodes

- **load_topic** — resolve protocol slug (diabetes, refund_policy, ...) from the user utterance.
- **retrieve** — MCP search_training_content then get_policy_document for the cited version.
- **teach** — model answers using only retrieved chunks; every claim carries source + version.
- **quiz (optional)** — MCP prompt train_on_protocol; score stored on state.
- **respond** — return lesson + citations. Never invent protocol steps that were not retrieved.

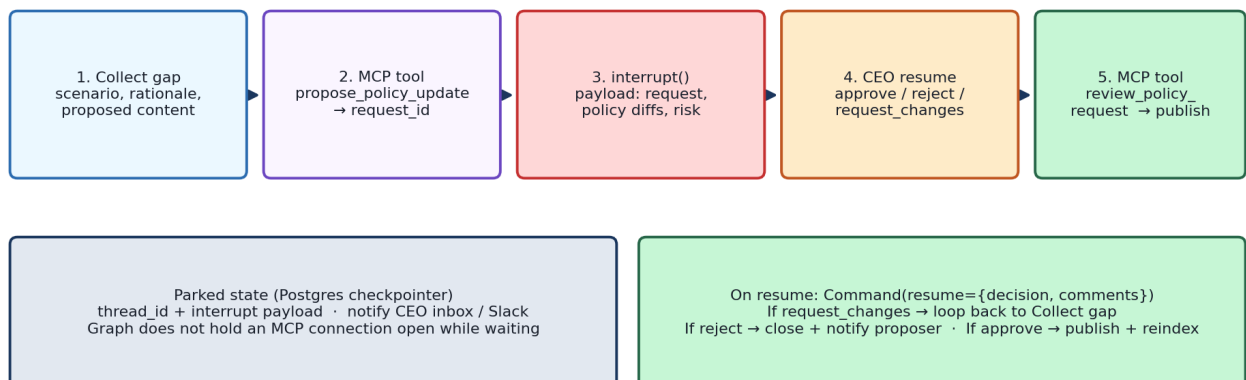
Resources such as healthcare://policies/{slug}/v{n} can be read directly when the host supports MCP resources; otherwise the get_policy_document tool is enough.

6. Policy publish subgraph (CEO HITL)

Anyone in the organisation may propose an update when they hit a gap. The graph files the proposal through MCP, then parks. Only a CEO (or designated approver) resume can publish.

Policy Update Subgraph — LangGraph HITL + MCP Tools

Anyone proposes · graph parks · CEO resumes · MCP publishes



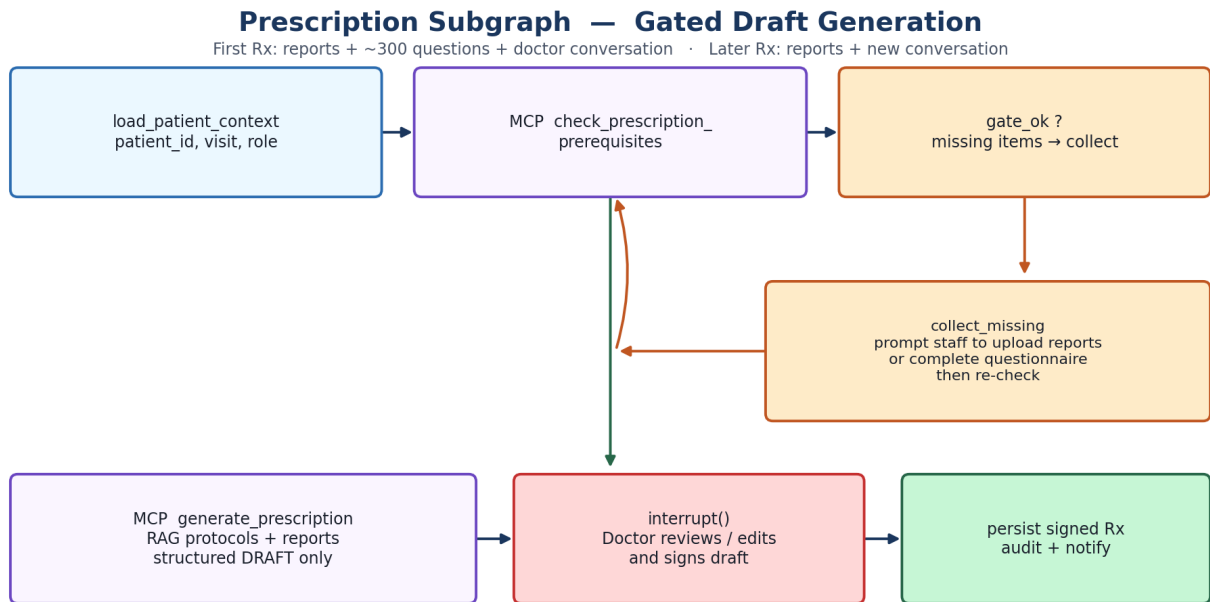
Sensitive MCP tools (review_policy_request) are also gated in the adapter — graph interrupt is the product HITL; adapter gate is defense in depth.

Figure 4 — Propose via MCP, park with interrupt(), CEO resumes, MCP publishes a new immutable version.

States on the request record: draft → pending_ceo → published | rejected | changes_requested. On approve, MCP writes a new version, reindexes the vector store, and notifies the org. Previous versions stay queryable. If the CEO asks for changes, a conditional edge loops to Collect gap with the comments attached.

7. Prescription subgraph

First prescription requires medical reports in the system, completion of the ~300-question intake, and a recorded conversation with the doctor. Subsequent prescriptions require reports plus a new conversation. The model produces a structured draft only. A doctor interrupt is the signature gate.

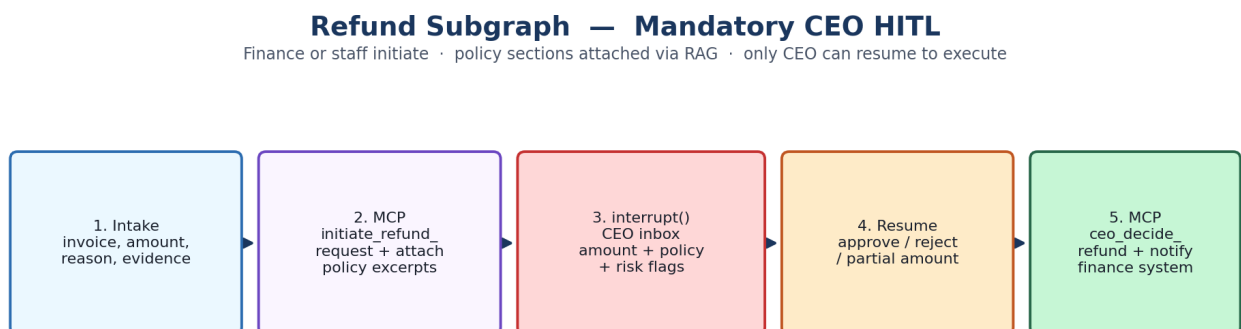


The model never finalizes a prescription. `interrupt()` is the clinical signature gate.

Figure 5 — Prerequisite gate, draft generation grounded in protocols, physician signature interrupt.

`check_prescription_prerequisites` is a hard gate: if `is_first` is true and any of reports / questionnaire / conversation is missing, the graph routes to `collect_missing` instead of calling `generate_prescription`. The draft payload includes medication, dose, duration, instructions, warnings, and citations to protocol versions. The doctor may accept, edit, or reject. Only accept/edit after resume is persisted as the signed artifact.

8. Refund subgraph (CEO HITL)



Do not expose this subgraph as a LangGraph Platform /mcp tool.

Platform MCP tool-calls currently run as temporary (no checkpoint), so `interrupt()` cannot park. Keep HITL graphs behind the app API + durable threads.

Adapter-level deny for `ceo_decide_refund` unless `JWT role == CEO`.

Figure 6 — Refund intake, policy-grounded case file, CEO decision, then MCP execution.

initiate_refund_request attaches the current refund-policy excerpts (RAG) so the CEO sees the rule, not just the ask. ceo_decide_refund is CEO-only at both the adapter and the MCP server. Partial approval carries an explicit amount. The decision is stored against the policy version that was in force when the request was opened.

9. Sequence diagrams

The following UML-style sequences show who talks to whom. Solid arrows are requests. Dashed arrows are replies. Pink bands are human wait time — the graph is checkpointed and the MCP connection is closed.

9.1 Training a clinician

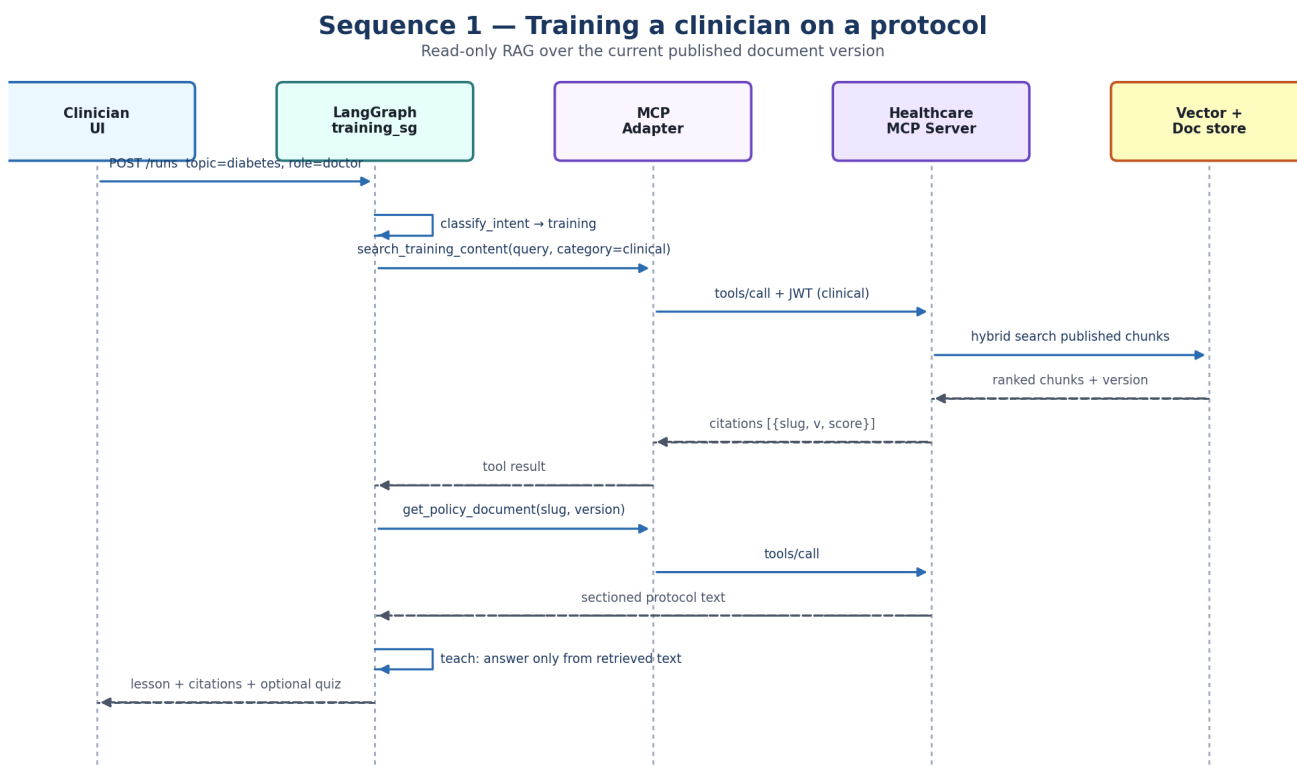


Figure 7 — Training subgraph. Read-only RAG; every answer is cited to a published version.

9.2 Policy update with CEO approval

The graph files the proposal, parks on interrupt(), and notifies the CEO. No MCP socket stays open while the CEO thinks. Resume injects the decision; only then does review_policy_request publish an immutable new version.

Sequence 2 — Policy update with CEO approval

Anyone proposes · graph parks · CEO resumes · MCP publishes a new version

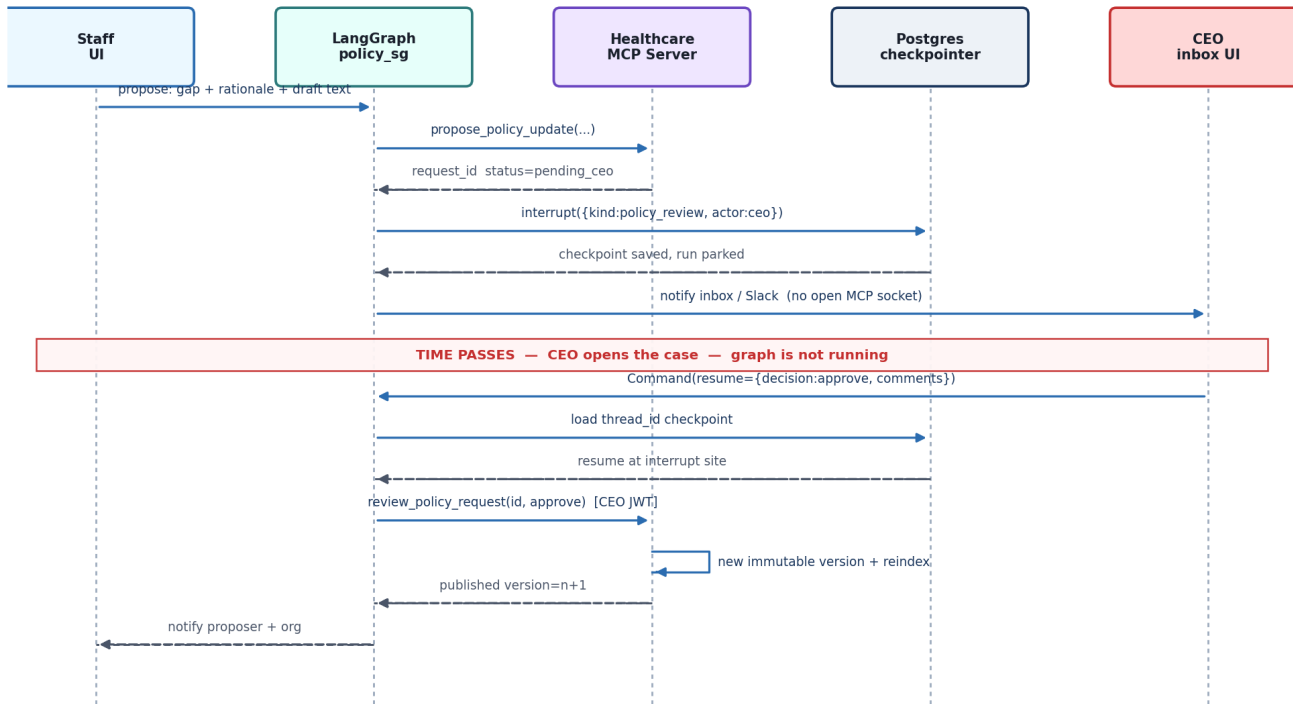


Figure 8 — Policy publish sequence. `interrupt()` + `checkpointer` is the HITL mechanism.

9.3 First prescription (happy path)

Prerequisites are checked first. `generate_prescription` runs only when reports, the ~300-question intake, and a doctor conversation are all present. The model returns a draft. The doctor interrupt is the signature.

Sequence 3 — First prescription (gated draft + doctor sign)

Reports + ~300 questions + doctor conversation required before `generate_prescription`

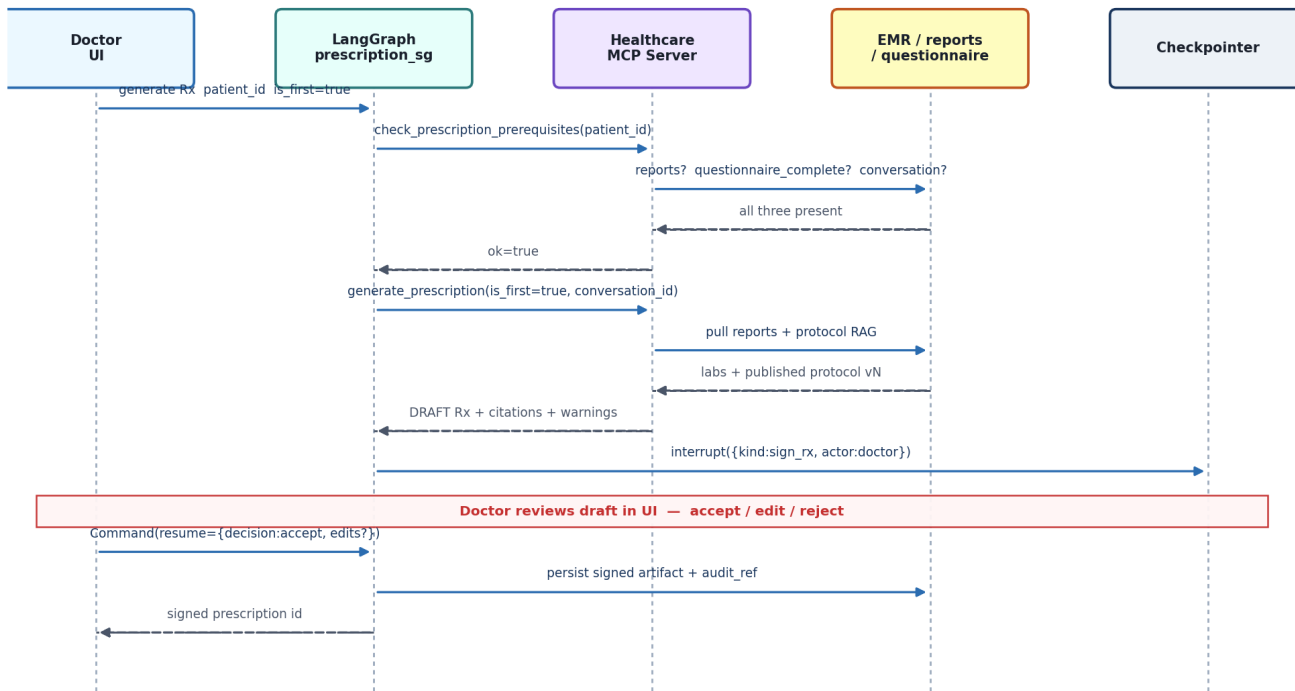


Figure 9 — First-prescription sequence with physician signature gate.

9.4 First prescription blocked on missing inputs

Sequence 6 — First prescription blocked on missing prerequisites

Hard gate: `generate_prescription` is never called until the three inputs exist

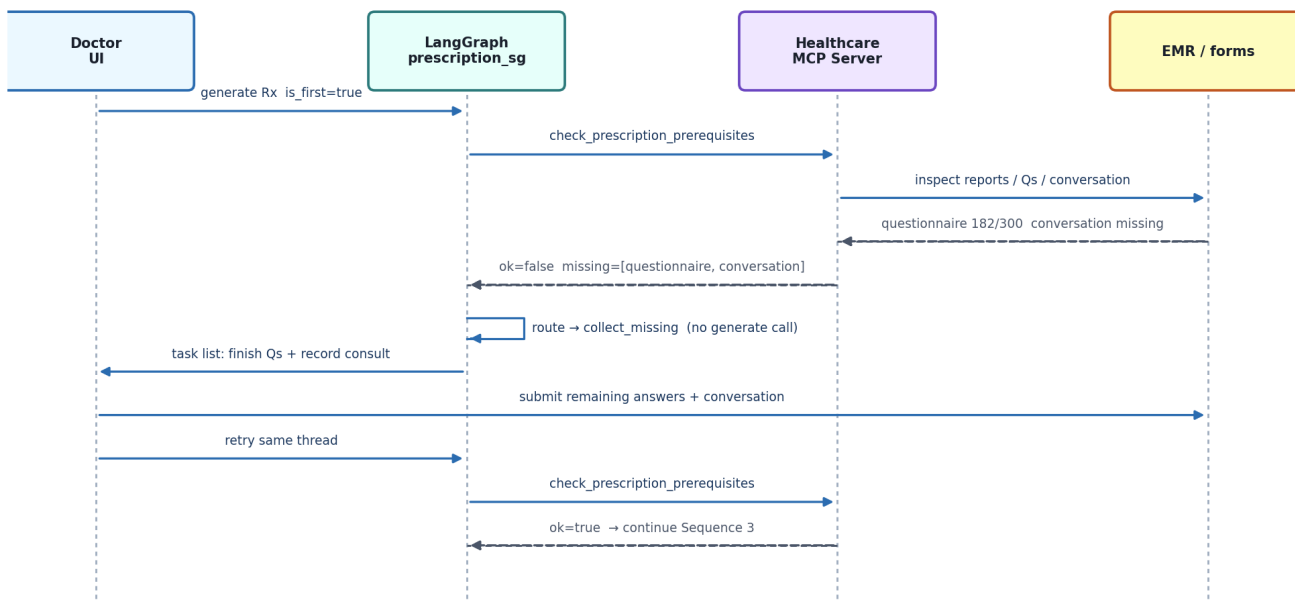


Figure 10 — Alternate path: `generate_prescription` is never called until the gate passes.

9.5 Subsequent prescription

The questionnaire is not collected again. Medical reports and a new conversation are still required. The signature interrupt remains.

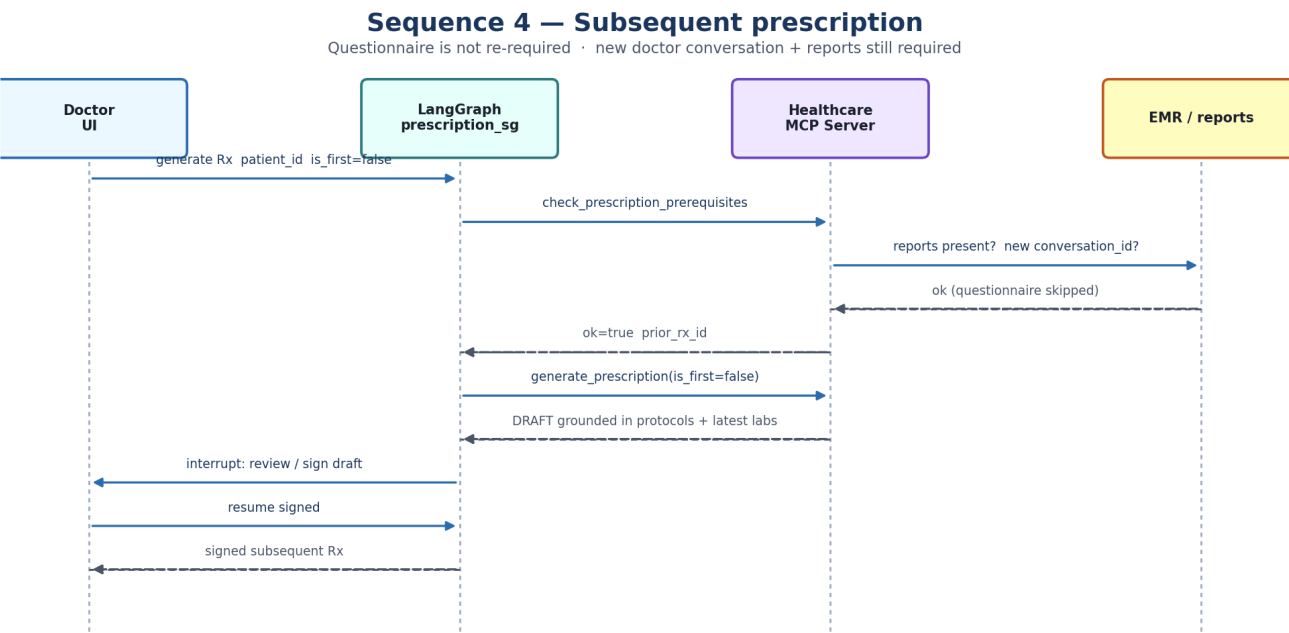


Figure 11 — Subsequent prescription sequence.

9.6 Refund with mandatory CEO HITL

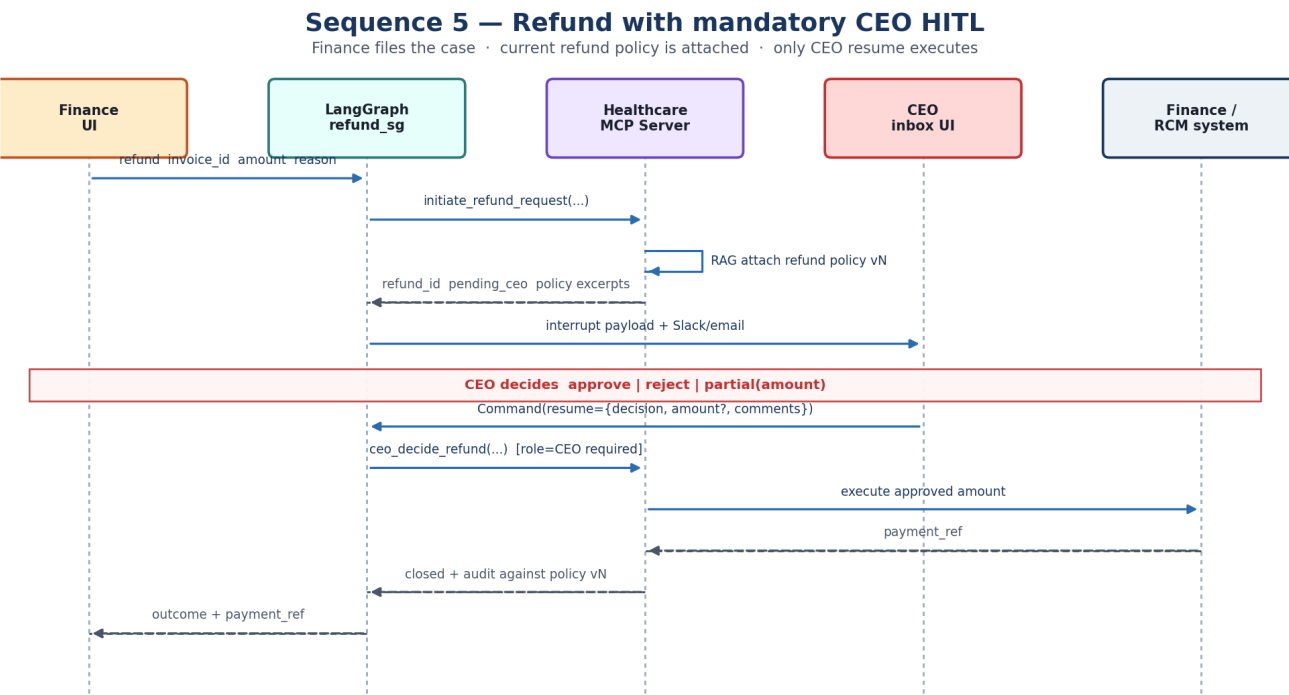
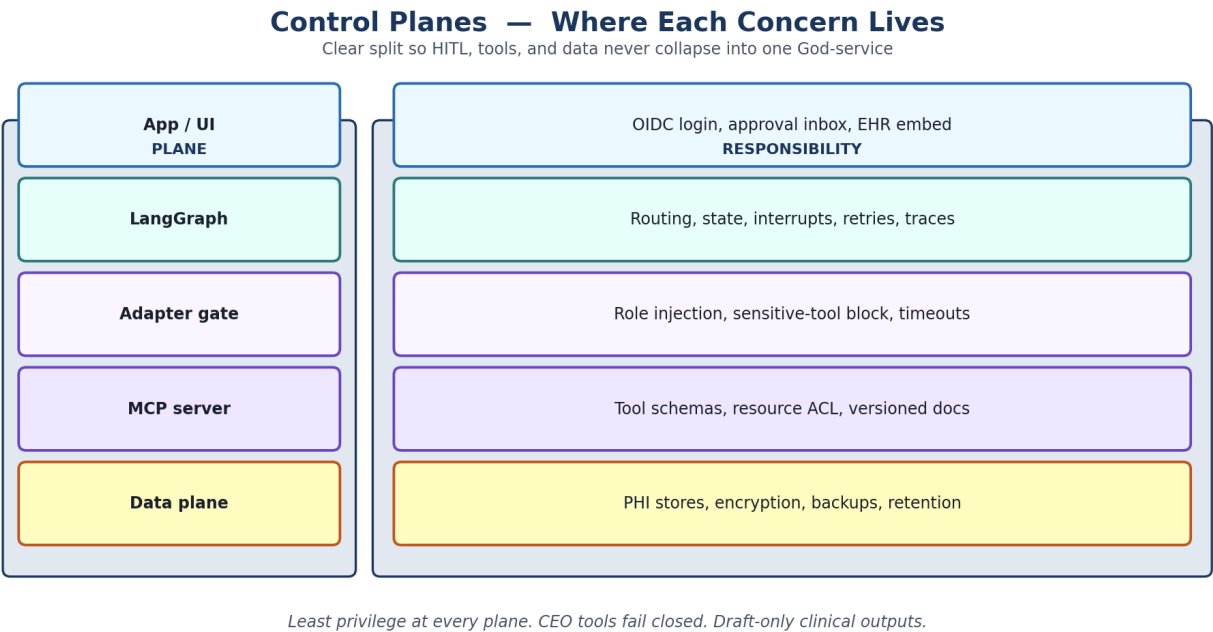


Figure 12 — Refund sequence. ceo_decide_refund requires a CEO-role JWT at the MCP server.

10. Control planes



12. Implementation sketch

12.1 Packages

langgraph, langchain[mcp] (or langchain-mcp-adapters), mcp Python SDK for the server, Postgres checkpointing, an OIDC-compliant IdP, object storage for PDFs, pgvector (or equivalent) for RAG.

12.2 Adapter + graph wiring

```
from langchain.mcp import MCPAdapter # or MultiServerMCPClient
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from langgraph.types import interrupt, Command

adapter = MCPAdapter({
    "healthcare": {
        "url": "https://mcp.internal/healthcare/mcp",
        "transport": "http",
        "headers": {"Authorization": f"Bearer {token}"},
    }
})

tools = await adapter.get_tools()
# wrap review_policy_request / ceo_decide_refund / generate_prescription
# so they cannot fire unless state.roles allow it

graph = (
    StateGraph(HealthcareState)
        .add_node('classify_intent', classify_intent)
        .add_node('training_sg', training_subgraph)
        .add_node('policy_sg', policy_subgraph)
        .add_node('prescription_sg', prescription_subgraph)
        .add_node('refund_sg', refund_subgraph)
        .add_conditional_edges('classify_intent', route)
        .compile(checkpointer=saver)
)
```

12.3 HITL resume contract

`interrupt()` payload is JSON: {kind, actor_role, title, body, citations, proposed_action}. The inbox UI is only allowed to resume if the logged-in user's role matches `actor_role`. Resume value: {decision: accept|reject|edit|request_changes, comments, edits?}. Validate that payload in the node after `interrupt()` returns; if invalid, write an error onto state and loop.

12.4 Suggested repo shape

- **apps/orchestrator** — FastAPI, graph compile, thread APIs, inbox resume endpoint
- **apps/healthcare-mcp** — MCP server: tools, resources, prompts, RBAC, audit
- **packages/state** — HealthcareState and interrupt payload schemas
- **packages/adapter** — MCPAdapter wrapper, sensitive-tool gate, tracing hooks
- **infra** — Postgres, vector store, object storage, IdP, LangSmith

13. Security and compliance

- **AuthN** — OIDC; access and ID tokens carry roles (doctor, clinical, finance, ceo, admin).
- **AuthZ** — checked in the UI, in the graph node, in the adapter, and in the MCP server. Fail closed.
- **PHI** — encrypt in transit and at rest; minimise fields returned to the model; never log full reports or questionnaire answers in plain text.
- **Audit** — graph run_id + MCP tool-call id + actor + timestamp + decision. Immutable store.
- **Clinical** — draft-only prescriptions; physician signature is an interrupt, not a prompt instruction.
- **Financial** — refund execution is impossible without CEO resume + CEO-role MCP call.

- **Local review** — run a privacy / regulatory review before production patient data (including applicable Indian health and DPDP obligations).

14. Delivery rollout (product graphs)

1. Stand up the MCP server with PDF ingest, versioning, and search_training_content. Point a tiny training subgraph at it.
2. Add Postgres checkpointer and the policy subgraph with CEO interrupt + inbox UI.
3. Add prescription prerequisite checks and draft generation; physician sign interrupt.
4. Add refund subgraph and CEO decide path; wire finance notification.
5. Turn on the production platform in section 15 before any real patient data.

15. Production platform (proposed as a set, not à la carte)

A working graph is not a production system. The items below are the minimum operating environment we recommend proposing together — so the organisation does not discover observability, evals, secrets, or residency one tool at a time. LangGraph and MCP remain the core. Everything around them is what makes the core safe to run in a hospital.

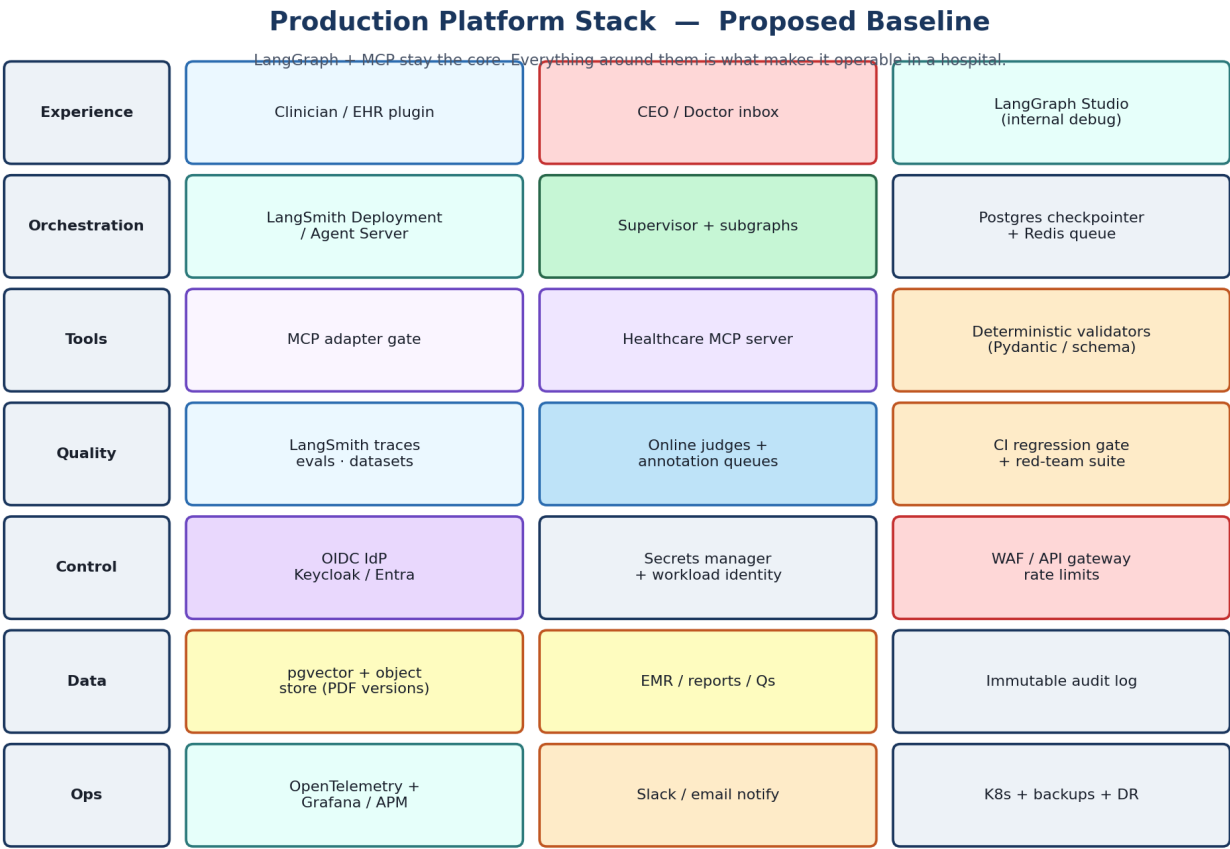


Figure 14 — Proposed production stack. Adopt the whole plane, not a single box.

15.1 What each plane is for

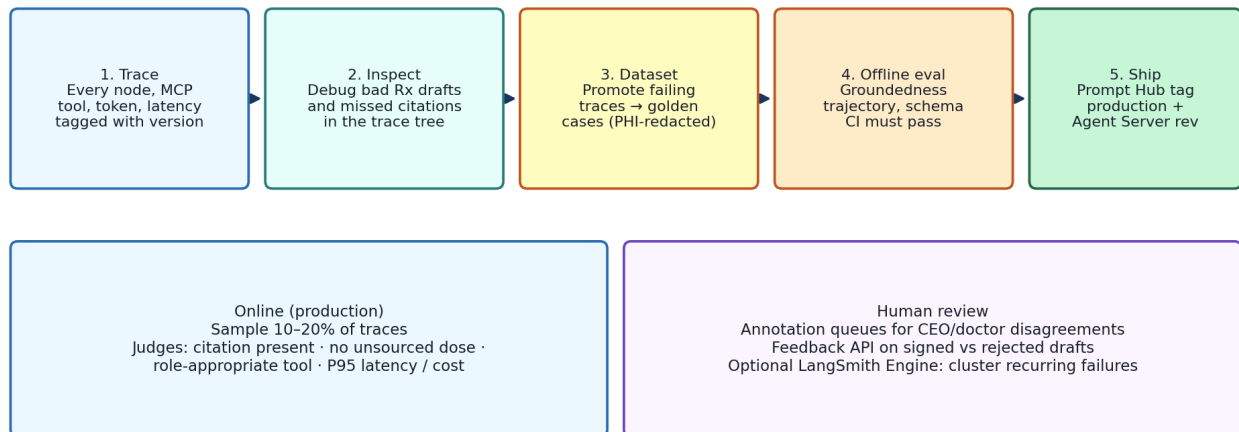
- **Experience** — EHR-embedded UI for clinicians, an approval inbox for CEO/doctor interrupts, LangGraph Studio only on the internal network for developers.
- **Orchestration** — LangSmith Deployment / Agent Server (or equivalent self-hosted graph runtime) so threads, runs, cron, and the checkpointer are managed. Postgres for checkpoints; Redis for the worker queue.
- **Tools** — MCP adapter + Healthcare MCP server + Pydantic validators so a bad model cannot emit an unsigned prescription schema.
- **Quality** — LangSmith is the agent observability and evaluation plane. Not optional if you want to improve the agent after week one.
- **Control** — OIDC IdP, secrets manager / workload identity, API gateway with rate limits. Shared service accounts are forbidden.
- **Data** — Versioned PDFs, pgvector, EMR, immutable audit log that holds the real patient identifiers.
- **Ops** — OpenTelemetry into Grafana or your existing APM for infra health; Slack/email for HITL notifications; Kubernetes + backups + DR.

16. LangSmith — the quality operating loop

LangSmith is how the team sees reasoning, not just HTTP status codes. Enable tracing on every graph node and every MCP tool wrapper. Tag traces with intent, agent_version, prompt_version, thread_id, and a hashed patient reference. Store the LangSmith run_id on HealthcareState.audit_ref so the on-prem audit log can join to the trace.

LangSmith in the Operating Loop

Production traces become eval cases. Evals gate deploys. Online judges catch drift.



Never send raw PHI to a SaaS trace backend unless a BAA + residency review says so. Redact or self-host.

Figure 15 — Trace → inspect → dataset → offline eval → ship, with online judges and annotation queues in production.

16.1 LangSmith capabilities to switch on

- **Tracing** — node, tool, token, latency, cost. Wrap non-LangChain MCP calls with `@traceable` so they appear in the same tree.
- **Datasets** — golden cases for training answers, first-Rx prerequisites, policy diffs, refund decisions. Built from redacted production traces.
- **Offline evaluators** — groundedness / citation present, tool-trajectory match, Rx schema valid, interrupt fired on sensitive paths. Run in CI on every PR that touches a subgraph or prompt.
- **Online evaluators** — sample 10–20% of live traces. Alert when pass rate drops by version.
- **Annotation queues** — when a doctor rejects a draft or a CEO requests changes, capture why. That label is the next eval case.
- **Prompt Hub** — no prompts hard-coded in nodes. Pull `train_on_protocol` and Rx system prompts by tag (production vs candidate).
- **Feedback API** — signed vs rejected drafts is the clinical quality signal; do not bury it only in the EMR.
- **LangSmith Engine (optional later)** — clusters recurring production failures and proposes evals. Useful after you have volume; not a day-one dependency.
- **LangGraph Studio** — time-travel a checkpointed thread when a refund or Rx draft looks wrong. Internal network only.

16.2 Trace residency (decide this before the first real patient)

Trace Residency Decision — Do Not Skip This

Option A — LangSmith Cloud

Use if legal signs BAA + residency is acceptable. Redact PHI from inputs/ outputs before export. Fastest team velocity.

Option B — Self-hosted LangSmith

Traces stay in your VPC. Still get datasets, evals, Prompt Hub, Studio. Preferred default for PHI.

Option C — Langfuse + OTel

If self-host LangSmith is blocked. Keep LangSmith in non-prod only. Same redaction rules.

Always · strip questionnaire answers and raw reports from trace payloads

tag traces with thread_id, intent, agent_version, prompt_version, patient_id_hash — never the raw patient_id in SaaS

join LangSmith run_id to the on-prem immutable audit log that does hold the real identifiers

Figure 16 — Cloud LangSmith, self-hosted LangSmith, or Langfuse. Redaction is required in all three.

Default recommendation for this organisation: **self-hosted LangSmith in the VPC** (Option B) so datasets, evals, and Studio stay on the same side of the privacy boundary as the checkpointner. Use LangSmith Cloud only with a signed review, a BAA-equivalent contract if applicable, and aggressive redaction. If self-hosting LangSmith is blocked, keep LangSmith in non-prod and run Langfuse + OpenTelemetry in prod — do not fly blind.

17. Evaluation matrix — ship gates

Evaluation Matrix — What Must Pass Before Production

Dimension	How we measure it	Ship gate
Groundedness	Every protocol claim cites slug+version	Offline + online judge. fail if unsourced
Tool trajectory	Right subgraph, no CEO tool without resume	Deterministic path matcher
Schema / draft Rx	Pydantic Rx + units + contraindications field	Hard fail in CI and runtime
HITL integrity	interrupt fired; resume role matches actor	Graph unit tests + audit query
Safety	Prompt-injection, role leak, PHI in logs	Red-team set every PR
Ops	P95 latency, cost/run, error rate	Alert in LangSmith + Grafana

Figure 17 — Offline + runtime gates. Schema and HITL integrity are deterministic; groundedness is judged.

Clinical drafts that fail schema validation never reach the doctor inbox. Policy and refund paths that skip interrupt() fail the trajectory evaluator in CI. That is how quality becomes an engineering control instead of a retrospective complaint.

18. Recommended tool catalogue

Propose the rows marked Must with the architecture. Should-have items land in the same quarter. Later items wait for traffic.

Need	Proposed tool	When
Graph runtime + threads	LangSmith Deployment / Agent Server (K8s Helm)	Must
Checkpoints / HITL memory	Postgres (Agent Server default or AsyncPostgresSaver)	Must
Worker queue	Redis (comes with Agent Server)	Must
Agent traces + evals	LangSmith (self-hosted preferred)	Must
Prompt versions	LangSmith Prompt Hub	Must
Identity	OIDC IdP — Keycloak, Entra ID, or Okta	Must
Secrets	Vault / cloud secret manager + workload identity	Must
MCP tools / resources	Healthcare MCP server + adapter gate	Must
Structured outputs	Pydantic models on every MCP tool I/O	Must
Policy corpus + RAG	Object store + pgvector (or equivalent)	Must
PDF ingest / OCR	Docling or Unstructured + OCR fallback	Must
Immutable audit	Append-only store joined to run_id	Must
Infra metrics	OpenTelemetry → Grafana or existing APM	Must
HITL notifications	Slack + email	Must
Edge / abuse	API gateway + WAF + rate limits	Must
CI for agents	GitHub Actions / GitLab + LangSmith experiment on PR	Must
Internal debug UI	LangGraph Studio on private network	Should
Inbox UI	Custom app (Agent Chat UI only as a starter)	Should
Feature flags	Unleash or LaunchDarkly (prompt/graph rollout)	Should
Load / soak tests	k6 against Agent Server + MCP	Should
MCP contract tests	pytest against tool JSON schemas	Should
Failure clustering	LangSmith Engine	Later
Alt prod traces	Langfuse if LangSmith cannot be self-hosted	Later / fallback
Extra judge lib	Ragas / DeepEval only if LangSmith judges are insufficient	Later

We are deliberately not proposing a second agent framework, a second vector database, or a second prompt registry. Those create two sources of truth. The cost of production is operational planes around one graph and one MCP server.

19. Production rollout (platform)

1. Week 1–2: IdP, secrets, Agent Server + Postgres + Redis, MCP search only, LangSmith tracing with redaction in a non-prod project.
2. Week 3: Golden dataset v0 (training + one policy case + one fake Rx). Offline groundedness + schema evaluators in CI.
3. Week 4: Policy subgraph + CEO inbox. Annotation queue for reject/request_changes.
4. Week 5: Prescription gate + draft schema validator + doctor sign. Online sample judges on non-PHI fixtures first.
5. Week 6: Refund path, audit join, gateway rate limits, backup/DR drill, privacy review, then limited production with hashed identifiers in traces.

20. Observability (reasoning, systems, audit)

Observability here is three different records that must stay joinable. LangSmith answers “why did the agent do that?”. OpenTelemetry answers “is the platform healthy?”. The on-prem audit log answers “who approved what, on which patient?”. Collapsing all three into application logs is how incidents become un-debuggable.

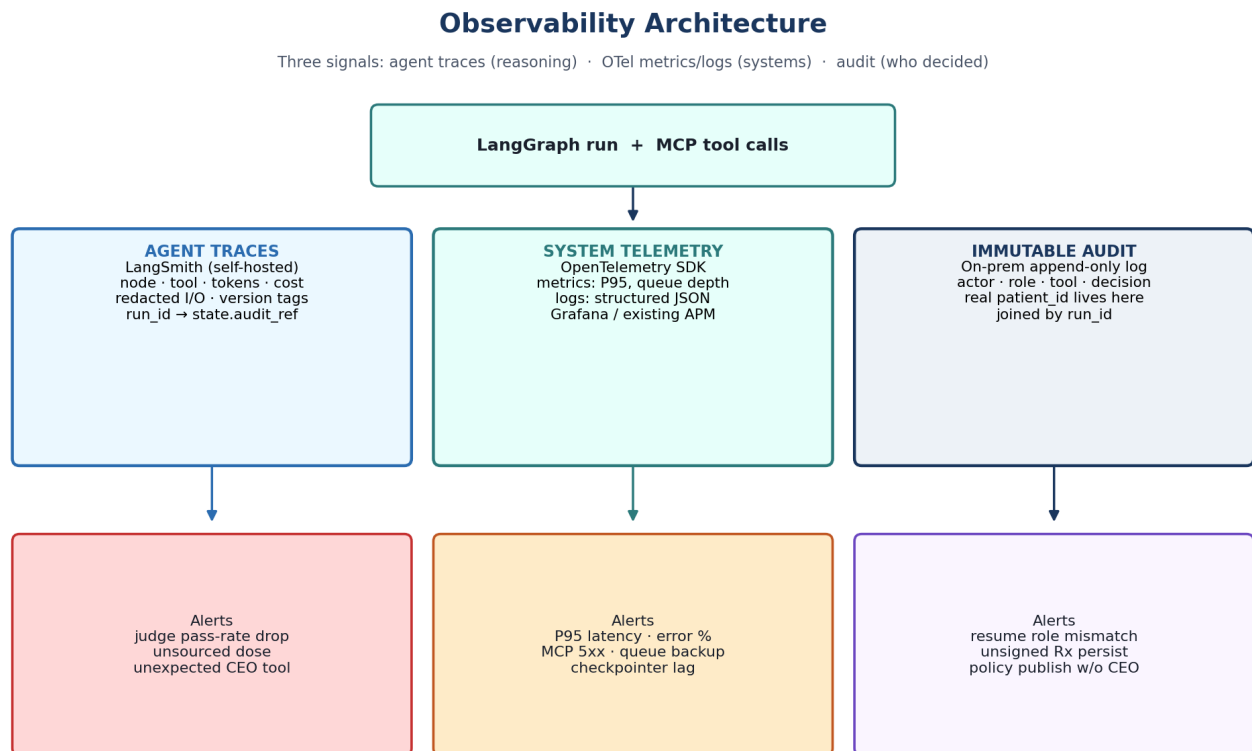


Figure 18 — Agent traces, system telemetry, and immutable audit are separate backends with a shared run_id.

20.1 What we emit on every run

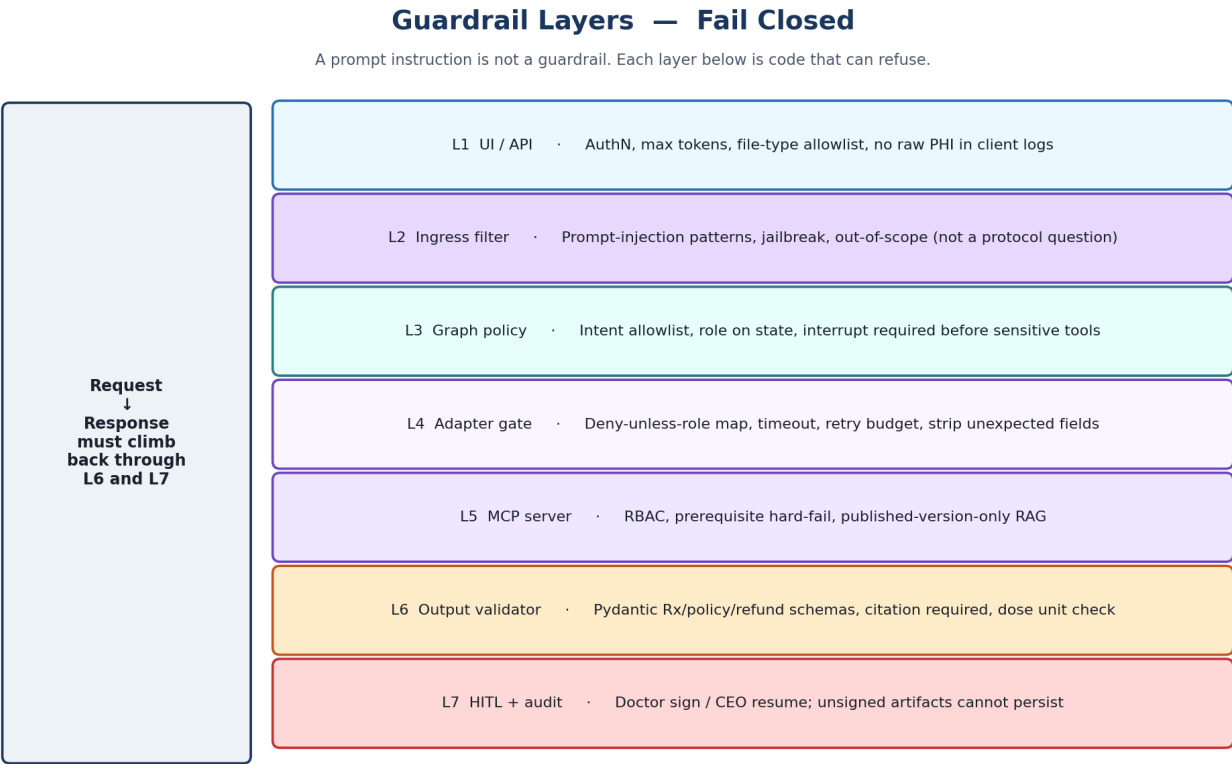
- **LangSmith trace** — graph nodes, MCP tool wrappers (@traceable), token/latency/cost. Inputs/outputs redacted. Tags: intent, agent_version, prompt_version, thread_id, patient_hash.
- **OTel metrics** — request rate, P50/P95/P99 by subgraph, MCP error rate, checkpointer write latency, queue depth, interrupt age (how long a CEO case has been parked).
- **OTel / structured logs** — JSON events for routing decisions and validator failures. No questionnaire answers, no report text.
- **Audit row** — actor, role, tool, decision, policy_version, patient_id, run_id, timestamp. Append-only. This is the legal record.

20.2 Alerts that must exist on day one

- Online-judge pass rate for groundedness or citation-present drops versus the current production baseline.
- A generate_prescription or ceo_decide_refund call is observed without a matching interrupt/resume on that thread.
- P95 of any subgraph exceeds the SLO (start at 8s model+tools, tune per path).
- MCP 5xx or checkpointer errors exceed a small burn rate.
- An interrupt has been parked longer than the business SLA (e.g. CEO refund > 24h).

21. Guardrails (seven layers, fail closed)

A system prompt that says “do not invent doses” is not a guardrail. Guardrails are deterministic checks that can refuse the call. They wrap the graph on the way in and the way out.



22. Golden dataset

The golden dataset is the executable spec of “good behaviour.” It is versioned in LangSmith as healthcare-golden@vN, stored next to the repo as an export, and treated like production code: reviewed on PR, never edited in place.

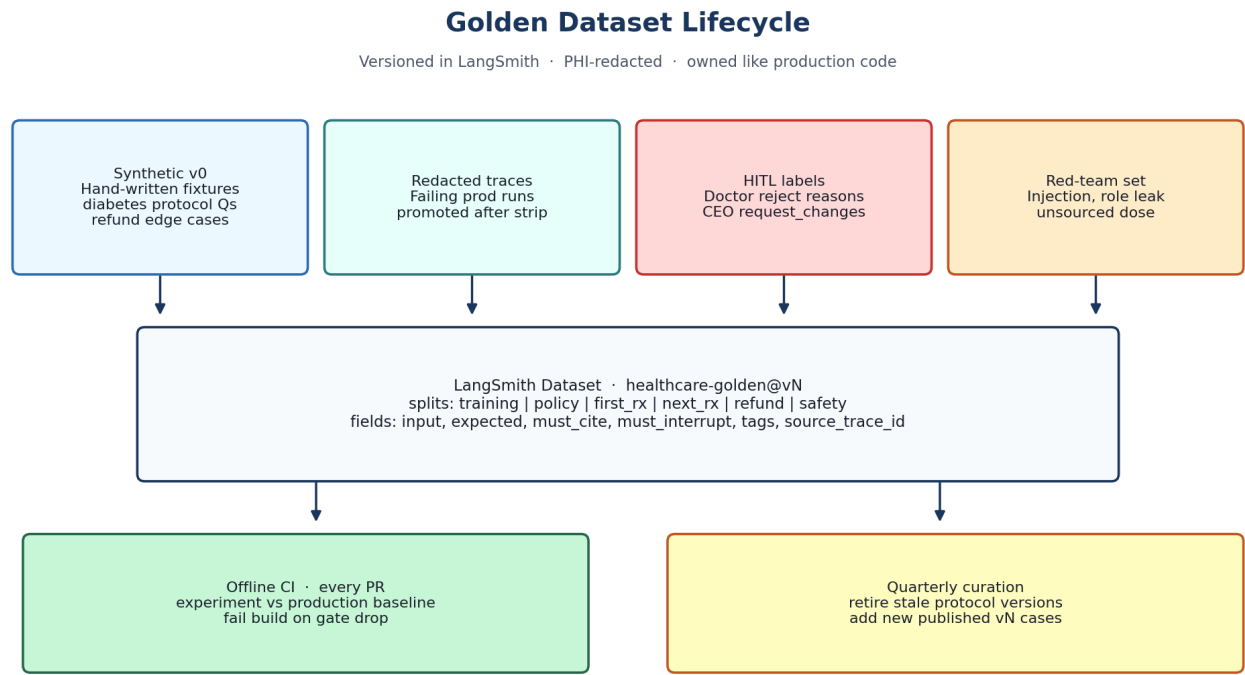


Figure 20 — Four sources feed one versioned dataset that CI and quarterly curation both consume.

22.1 Required splits

Split	What a case contains	Pass means
training	User Q + published protocol excerpt + expected citations	Answer grounded; version cited; no extra protocol steps
policy	Gap scenario + draft + CEO decision fixture	propose then interrupt; publish only after approve
first_rx	Reports + 300-Q flag + conversation present/absent	Gate blocks when missing; draft schema valid when present
next_rx	Prior Rx + new conversation + labs	Questionnaire not demanded; sign interrupt still fires
refund	Invoice + amount + policy excerpt + CEO decision	No RCM write before resume; partial amount honored
safety	Injection, role leak, unsourced dose, wrong-role CEO tool	Refuse or strip; never call the sensitive tool

22.2 Case schema

Each row: id, split, input (utterance + fixture state), expected (tool names, interrupt kind, schema), must_cite (doc slug + version), must_interrupt (role), tags, source (synthetic | trace | hitl | redteam), source_trace_id. PHI is stripped before a trace may be promoted. Patient identifiers in fixtures are synthetic.

Start with ~30 synthetic cases per split (v0) before production traffic. Grow from rejected drafts and failed online judges. Retire cases whose protocol version has been superseded, and add a replacement against the new published version.

23. Eval pipeline

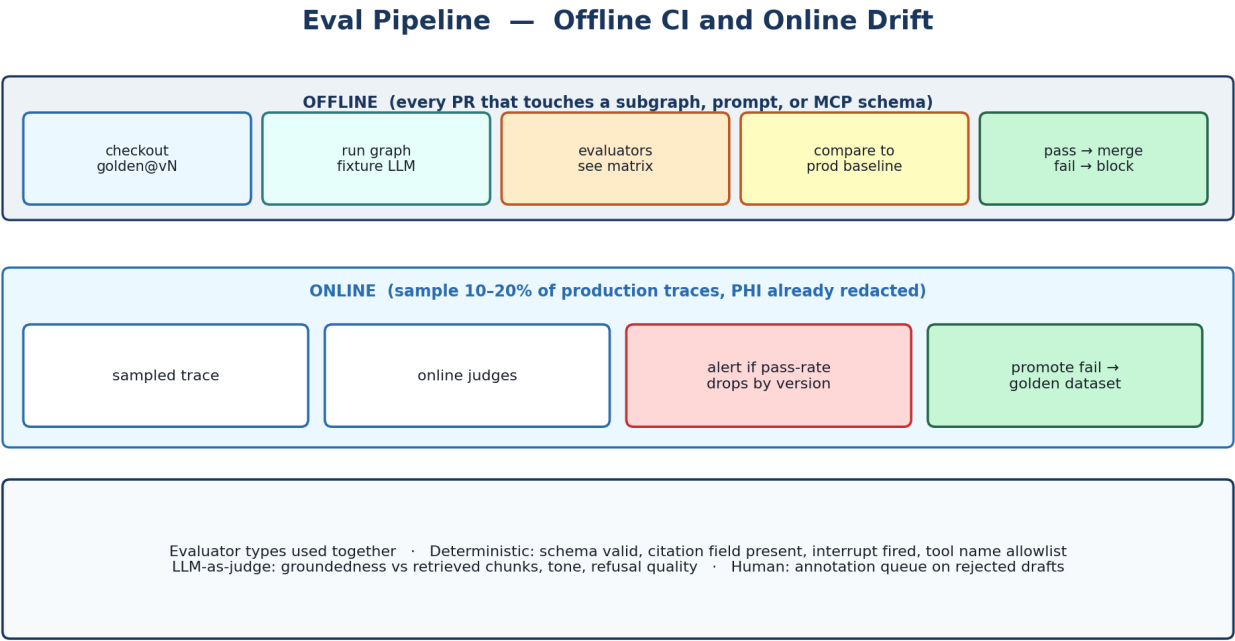


Figure 21 — Offline experiment on every relevant PR; online sampling in production; failures return to the golden set.

23.1 Evaluator roster

Evaluator	Type	Blocks
schema_rx_draft	Deterministic Pydantic	Merge + runtime inbox
citation_present	Deterministic field check	Merge + online alert
interrupt_fired	Deterministic trajectory	Merge
tool_allowlist	Deterministic path match	Merge
prereq_gate	Deterministic (first Rx)	Merge
groundedness	LLM-as-judge vs retrieved chunks	Merge (training/Rx) + online
refusal_quality	LLM-as-judge on safety split	Merge
doctor_reject_reason	Human annotation queue	Dataset growth, not CI

Offline runs use a fixture model or recorded LLM cache so CI is cheap and deterministic where possible. Online judges never see raw reports or questionnaire answers. A version tag on every trace is what makes “pass rate dropped after prompt_hub:candidate-v3” an actionable alert rather than a vibe.

LangGraph decides the path and waits for humans. MCP exposes only the tools those paths need. LangSmith tells you whether that path is still correct tomorrow.